E-Commerce Data Engineering Pipeline

1. Project Setup

The project was created as an end-to-end E-Commerce Data Engineering Pipeline. A structured project workspace was prepared to organize the source data, processing scripts, SQL files, screenshots, dashboard, and documentation.

The project structure includes separate folders for source data, incremental data, notebooks, SQL, screenshots, dashboard, and documentation.

The four datasets provided for the project are:

- customers.csv
- inventory.csv
- orders.csv
- order_items.csv

2. Source Dataset Inspection

The source datasets were loaded using Python and Pandas to understand their structure before starting the data processing pipeline.

The initial dataset details were:

Dataset	Rows	Columns
Customers	10,000	11
Inventory	5,000	9
Orders	50,150	10
Order Items	200,000	9

The column names and basic structure of each dataset were also inspected.

3. Source Data Quality Profiling

Initial data-quality checks were performed on all four datasets. The checks included missing values, duplicate records, and data types.

The following issues were identified:

Dataset	Data Quality Findings
Customers	No missing values or duplicate rows
Inventory	152 missing stock_quantity values
Orders	501 missing customer_id values and 150 duplicate rows
Order Items	2,986 missing quantity values and 4,956 missing line_total values

Several date and timestamp columns were also initially detected as object data types. These will be handled during the transformation stage.

The identified issues will be used later to design the Silver-layer cleaning and data-quality rules.

4. Project Configuration

A centralized config.py file was created to manage the project paths and dataset locations.

The configuration manages paths for:

- Source datasets
- Incremental datasets
- Notebooks
- SQL files
- Screenshots
- Dashboard
- Documentation

Centralizing the paths makes the project easier to maintain and avoids repeatedly writing hardcoded file locations in different scripts.

5. Landing Layer

A separate Landing layer was created to preserve the incoming source data before any cleaning or transformation is performed.

The Landing layer contains separate folders for:

landing/

├── customers/

├── inventory/

├── orders/

└── order_items/

The original source files in the data/source folder are kept unchanged.

This represents the first stage of the project's data pipeline and follows the Landing → Bronze → Silver → Gold architecture.

6. Landing Data Ingestion

A Python ingestion script named ingest_to_landing.py was implemented to copy the four source datasets from the source directory into their respective Landing folders.

The ingestion process:

- Checks whether each source file exists.
- Creates the required Landing directories.
- Copies the source files without modifying the original data.
- Records an ingestion timestamp.
- Displays the ingestion status for each dataset.

All four datasets were successfully ingested into the Landing layer.

This provides a controlled starting point for the next stage of the pipeline, where the data will be converted into the Bronze layer.

7. Bronze Layer

The Bronze layer was created to store the data received from the Landing layer while preserving the original records before applying cleaning or business transformations.

The four datasets were processed separately:

- Customers
- Inventory
- Orders
- Order Items

During Bronze ingestion, additional metadata columns were added to support data lineage and tracking:

- bronze_ingestion_timestamp
- source_file_name
- load_date

No data-quality corrections were performed at this stage. The original null values and duplicate records were retained so that they could be handled in the Silver layer.

The Bronze files were successfully created for all four datasets.

Bronze Layer Structure

bronze/

├── customers/

│ └── customers_bronze.csv

├── inventory/

│ └── inventory_bronze.csv

├── orders/

│ └── orders_bronze.csv

└── order_items/

    └── order_items_bronze.csv

This layer provides a stable intermediate stage between raw ingested data and the cleaned Silver layer.

8. Silver Layer – Data Cleaning and Transformation

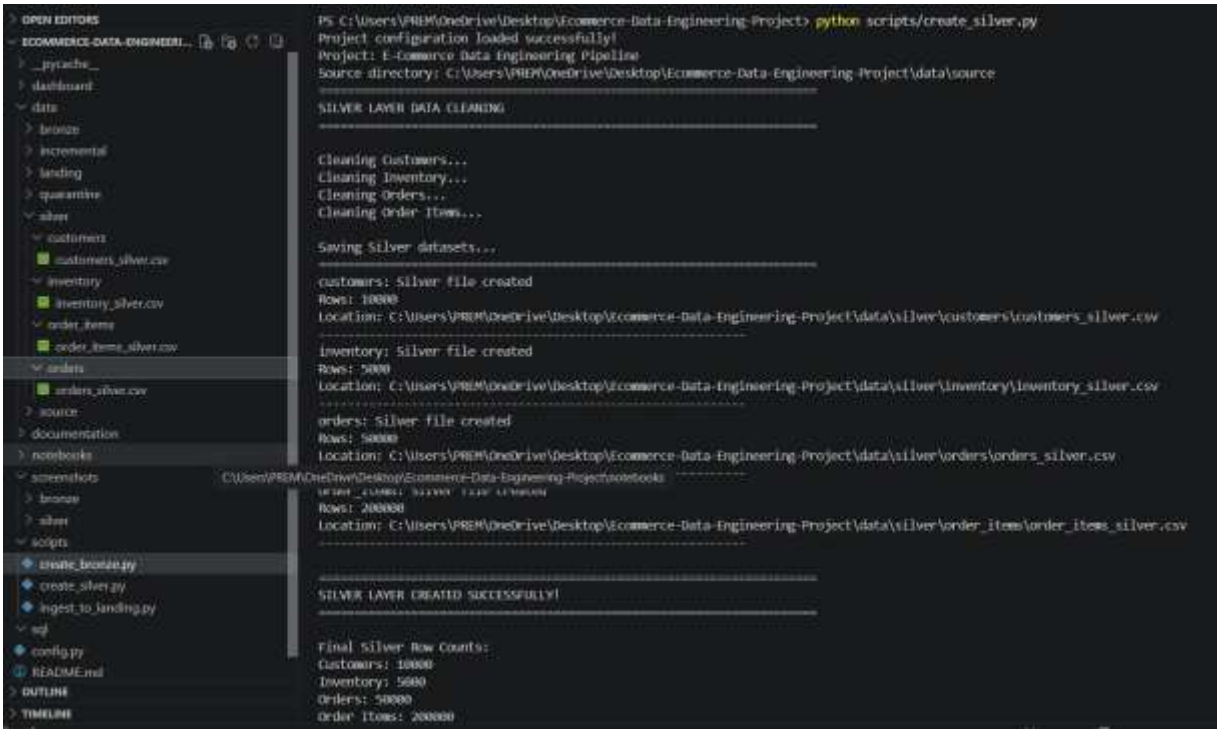
The Silver layer was created to clean and transform the data received from the Bronze layer.

The following cleaning and transformation operations were performed:

- Removed duplicate customer, inventory, order, and order-item records based on their respective identifiers.
- Converted date fields into proper datetime format.
- Converted numeric columns into appropriate numeric data types.
- Standardized text fields by removing unnecessary spaces.
- Converted customer email addresses to lowercase.
- Handled missing inventory stock quantities by assigning a default value of zero.
- Recalculated missing order-item line_total values using quantity and unit price wherever possible.
- Cleaned and standardized important identifier and categorical columns.

The cleaned datasets were then saved as separate Silver files for customers, inventory, orders, and order items.

The Silver layer provides cleaner and more consistent data for the validation, quarantine, Delta MERGE, and Gold-layer processing stages.



9. Data Quality Validation and Quarantine

A separate data-quality validation stage was implemented after Silver-layer cleaning. The purpose was to identify records that still violated important data-quality or business rules and separate them from trusted records.

The validation included checks for:

- Missing customer IDs
- Missing or invalid order IDs
- Invalid order dates
- Negative order amounts
- Missing or invalid quantities
- Missing SKU IDs
- Missing or negative unit prices
- Missing line totals
- Invalid inventory quantities

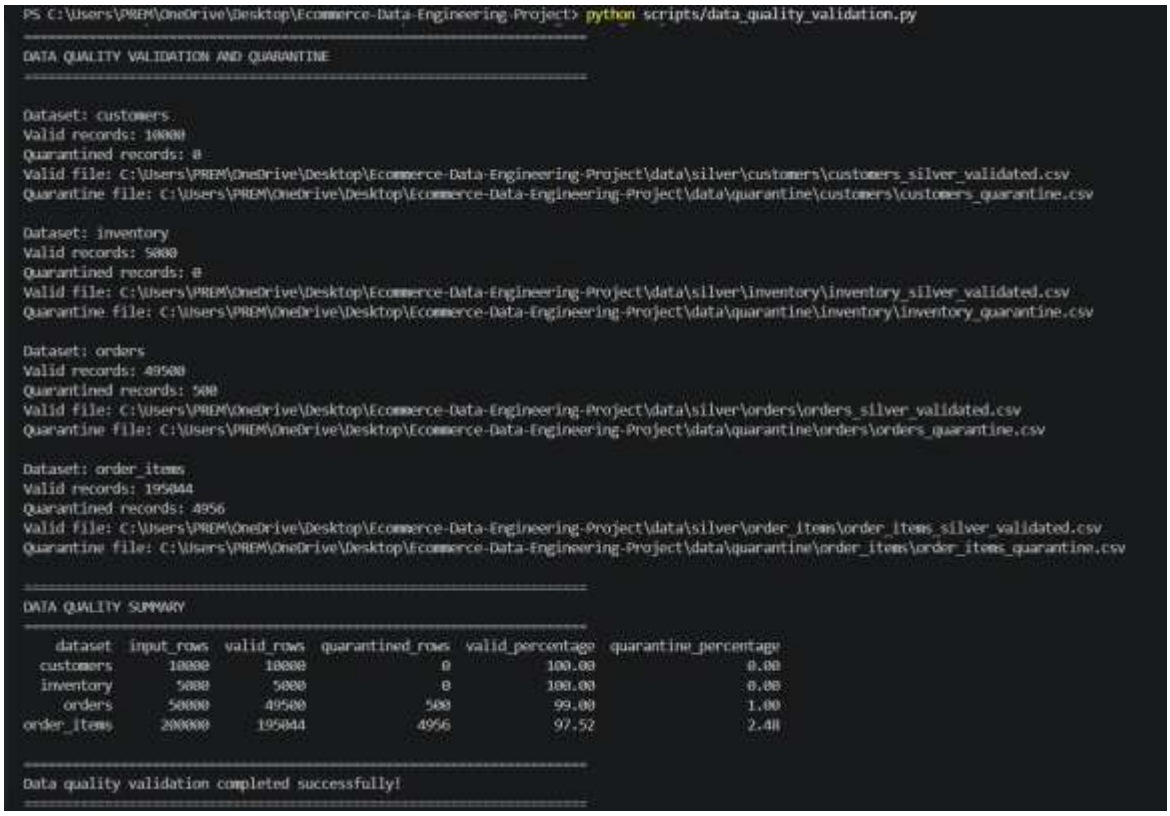
Records that failed validation were not directly used for further processing. Instead, they were stored in separate quarantine files along with a quarantine_reason column.

Validation Results

Dataset	Input Records	Valid Records	Quarantined Records	Quarantine Rate
Customers	10,000	10,000	0	0%
Inventory	5,000	5,000	0	0%
Orders	50,150	49,500	650	1.30%
Order Items	200,000	195,044	4,956	2.48%

The validated records were saved separately for trusted Silver processing, while rejected records were stored in the quarantine layer for further investigation.

This approach prevents invalid records from silently entering downstream analytics and provides better data traceability and quality control.



10. Referential Integrity Validation

Referential integrity checks were performed to verify relationships between the major datasets before downstream processing.

The following relationships were validated:

- orders.customer_id → customers.customer_id
- order_items.order_id → orders.order_id
- order_items.sku_id → inventory.sku_id

Results

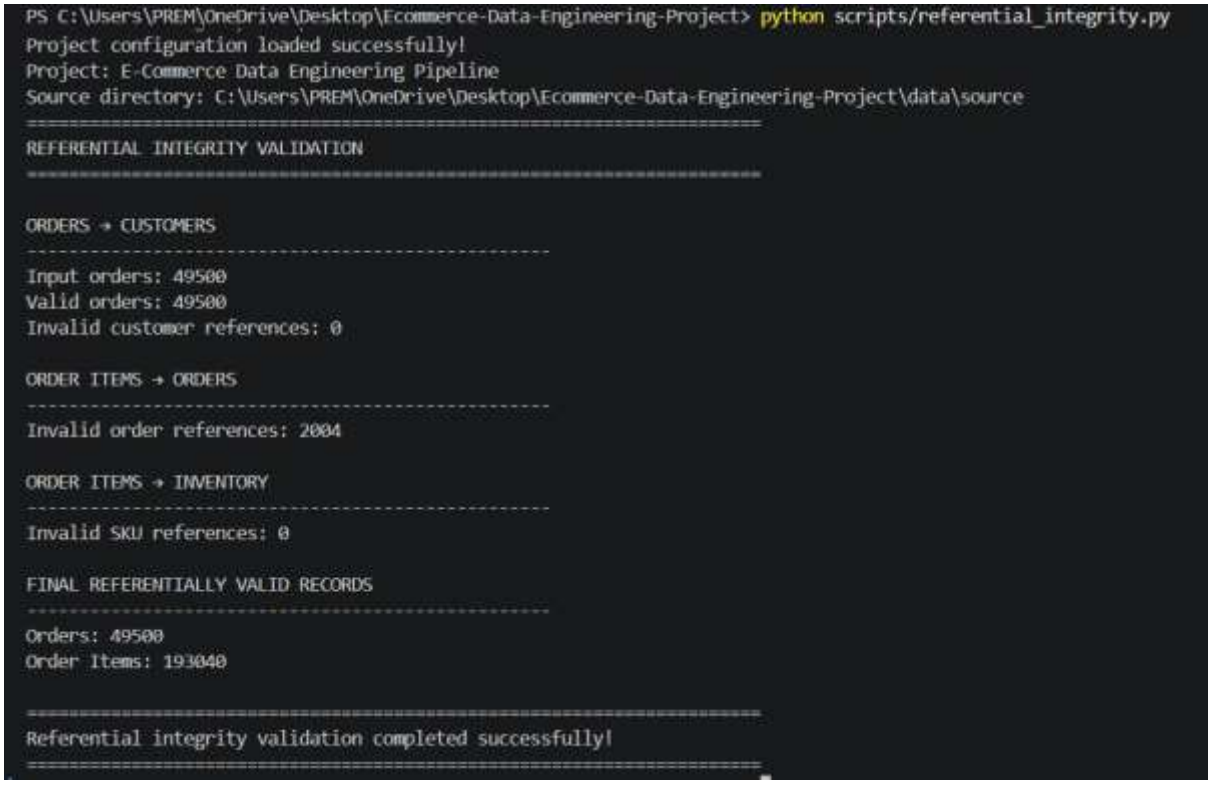
Relationship	Result
Orders → Customers	49,500 valid references
Order Items → Orders	2,004 invalid references
Order Items → Inventory	0 invalid references

All 49,500 validated orders had a matching customer record, and all order items had valid SKU references in the inventory dataset.

However, 4,956 order items referenced order IDs that were not available in the validated Orders dataset. These records were separated into the quarantine layer instead of being used in downstream analytics.

After referential integrity validation, 193,040 order-item records remained valid.

This step ensures that downstream joins and analytics are performed only on records with valid relationships between the datasets.



11. Gold Layer – Daily Revenue

The Gold layer was started to create business-ready analytical datasets from the validated Silver data.

The first Gold dataset created was **daily_revenue**.

The daily revenue calculation combines the validated Orders and Order Items datasets using order_id.

The following processing was performed:

- Joined Orders with Order Items using order_id.
- Excluded cancelled orders from revenue calculations.
- Converted the order timestamp into a calendar date.
- Grouped the data by:
    - Date
    - Region
    - Category
- Calculated total revenue.
- Calculated the number of distinct orders.
- Calculated average order value (AOV).

The resulting Gold dataset was saved as:

data/gold/daily_revenue.csv

Daily Revenue Output

Metric	Result
Gold rows	121,557
Total revenue	6,395,746,625.20
Total orders	121,557

The output file was successfully created and verified.

Example output:

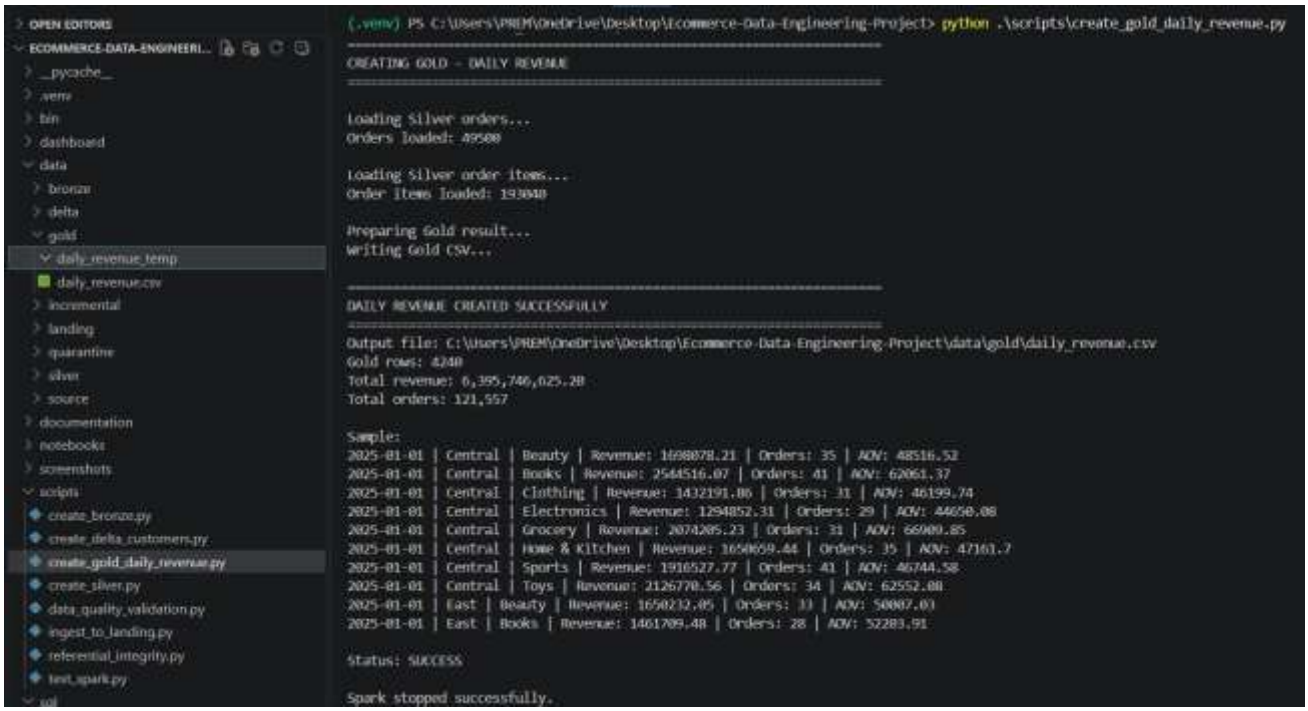
2025-01-01 | Central | Beauty

Revenue: 1,698,078.21

Orders: 35

AOV: 48,516.52

This dataset provides a business-ready view of revenue performance by date, region, and product category.



12. Gold Layer – Fulfillment KPI

The second Gold dataset created was **fulfillment_kpi**.

The purpose of this dataset is to provide fulfillment performance metrics grouped by **date, warehouse, and region**.

The validated Silver Orders dataset was used as the source.

The following processing was performed:

- Loaded the validated Silver Orders data.
- Converted order_date into a proper date.
- Standardized order status values to lowercase.
- Grouped orders by:
    - Date
    - Warehouse
    - Region
- Calculated total orders.
- Calculated delivered orders.
- Calculated cancelled orders.
- Calculated shipped orders.

- Calculated delivery rate.
- Calculated cancellation rate.
- Calculated shipment rate.

The resulting Gold dataset was saved as:

data/gold/fulfillment_kpi.csv

Fulfillment KPI Output

Metric	Result
Gold rows	2,650
Total orders	49,500
Delivery rate	20.52%
Cancellation rate	20.42%
Shipment rate	20.48%

Example output:

2025-01-01 | Warehouse: WH-BLR | Region: Central

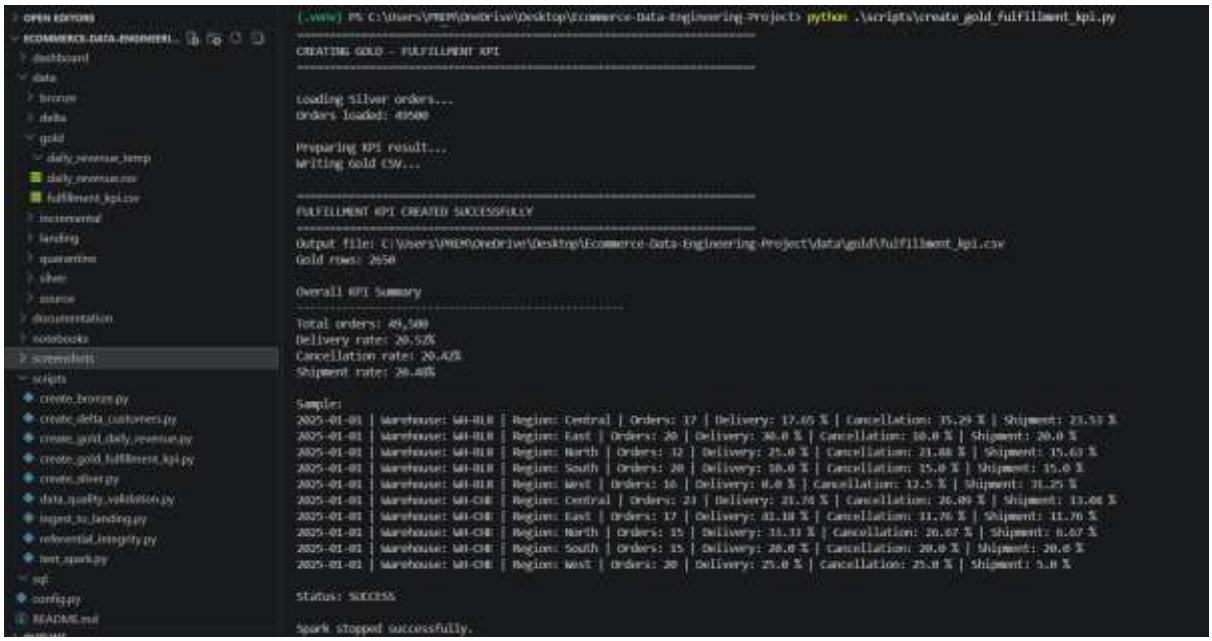
Orders: 17

Delivery: 17.65%

Cancellation: 35.29%

Shipment: 23.53%

The Gold Fulfillment KPI dataset was successfully created and provides a business-ready view of order fulfillment performance across dates, warehouses, and regions.



13. Gold Layer – Inventory Health

The third Gold dataset created was **inventory_health**.

The purpose of this dataset is to provide a business-ready view of inventory availability, stock levels, reorder requirements, and SKU demand.

The validated Silver Inventory and Order Items datasets were used as inputs.

The following processing was performed:

- Loaded the validated Silver Inventory data.
- Loaded the validated Silver Order Items data.
- Calculated demand for each SKU from the available validated order-item history.
- Joined SKU demand information with inventory records.
- Classified each SKU into a stock status:
    - stockout
    - below_reorder
    - healthy
    - overstock
- Created a reorder flag for SKUs requiring replenishment.
- Calculated inventory value using stock quantity and unit cost.
- Created a demand_30_day metric from the available order-item demand data.

The resulting Gold dataset was saved as:

data/gold/inventory_health.csv

Inventory Health Results

Metric	Result
Total SKUs	5,000
Stockout	154
Below reorder	144
Healthy	270
Overstock	4,432
SKUs requiring reorder	298
Total inventory value	36,597,321,257.46

Example:

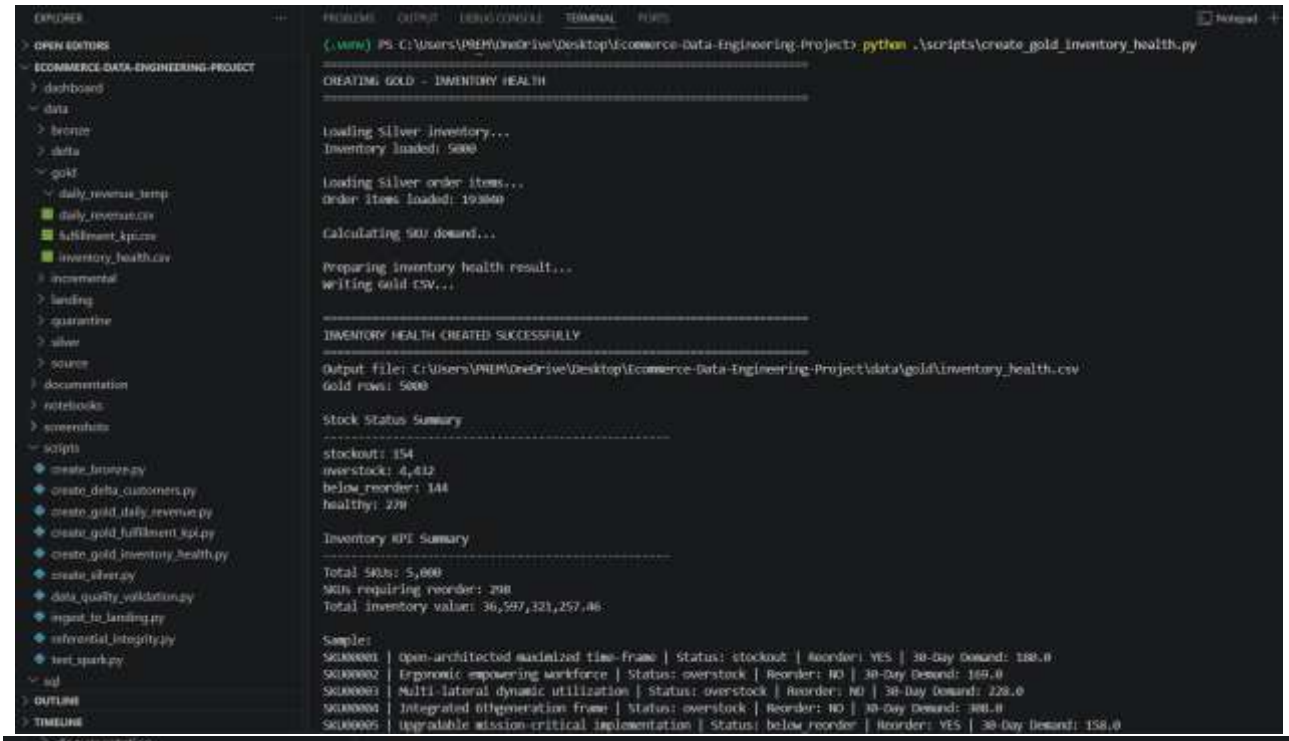
SKU00001 | Open-architected maximized time-frame

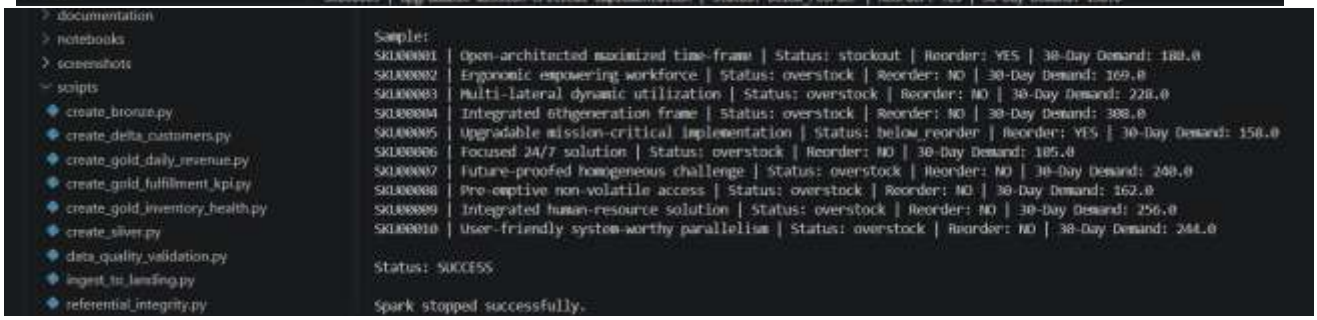
Status: stockout

Reorder: YES

30-Day Demand: 180

The Inventory Health Gold dataset was successfully created and provides an analytical view of current inventory conditions and replenishment requirements.





14. Gold Layer – Customer LTV

The fourth Gold dataset created was **customer_ltv**.

The purpose of this dataset is to provide customer-level lifetime value and customer segmentation metrics.

The validated Silver Customers and Orders datasets were used as inputs.

The following processing was performed:

- Loaded the validated Silver Customers data.
- Loaded the validated Silver Orders data.
- Excluded cancelled orders from customer spending calculations.
- Calculated lifetime spend for each customer.
- Calculated order frequency using distinct order IDs.
- Identified each customer's latest order date.
- Calculated customer recency in days.
- Joined the calculated customer metrics with customer information.
- Classified customers into four segments based on lifetime spend:
    - VIP
    - High Value
    - Mid Value
    - Low Value

The resulting Gold dataset was saved as:

data/gold/customer_ltv.csv

Customer LTV Results

Metric	Result
Total customers	10,000
Active customers	9,829
Active customer rate	98.29%
Average LTV	98,757.09
VIP	4,404
High Value	3,497
Mid Value	1,507

Metric	Result
Low Value	592

Example:

CUST000002 | Arunima Ahuja

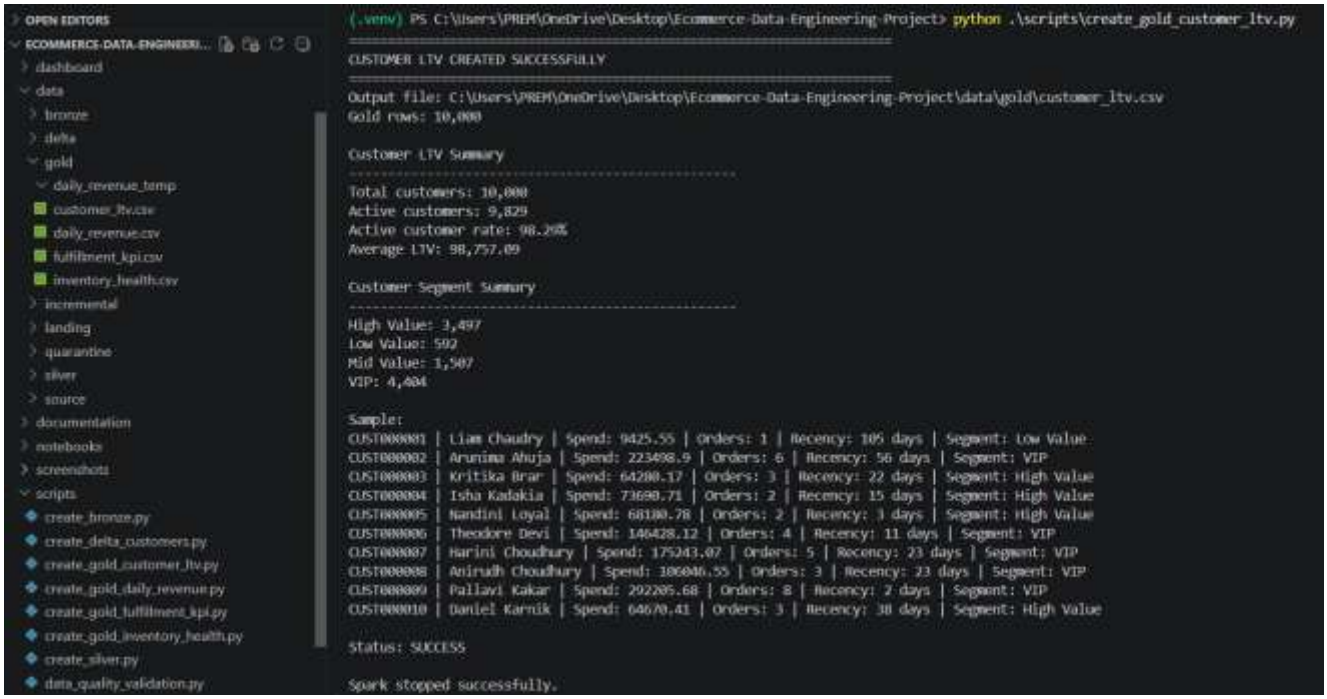
Spend: 223,498.90

Orders: 6

Recency: 56 days

Segment: VIP

The Customer LTV Gold dataset was successfully created and provides a business-ready customer-level view of lifetime spending, order frequency, recency, and customer value segments.



15. Gold Layer – Reconciliation and Audit

A reconciliation stage was implemented after the Gold layer to verify that data was successfully transferred through the pipeline and to provide visibility into data-quality outcomes.

Two reconciliation files were created:

- reconciliation_row_counts.csv
- reconciliation_dq_summary.csv

Row Count Reconciliation

The row-count reconciliation captures the number of records available at each pipeline layer.

Layer	Dataset	Rows
Bronze	Customers	10,000
Bronze	Inventory	5,000
Bronze	Orders	50,150
Bronze	Order Items	200,000
Silver	Customers	10,000
Silver	Inventory	5,000
Silver	Orders	49,500
Silver	Order Items	195,044
Gold	Daily Revenue	4,240
Gold	Fulfillment KPI	2,650
Gold	Inventory Health	5,000
Gold	Customer LTV	10,000

This reconciliation provides an audit trail of the records processed through the Bronze, Silver, and Gold layers.

Data Quality Reconciliation

The DQ reconciliation compares Bronze records with trusted Silver records and quarantined records.

Dataset	Bronze Records	Silver Records	Quarantined	Pass Rate	Quarantine Rate
Orders	50,150	49,500	650	98.70%	1.30%
Order Items	200,000	195,044	4,956	97.52%	2.48%

The reconciliation confirms that invalid Order and Order Item records were separated from trusted Silver data instead of silently entering downstream analytics.

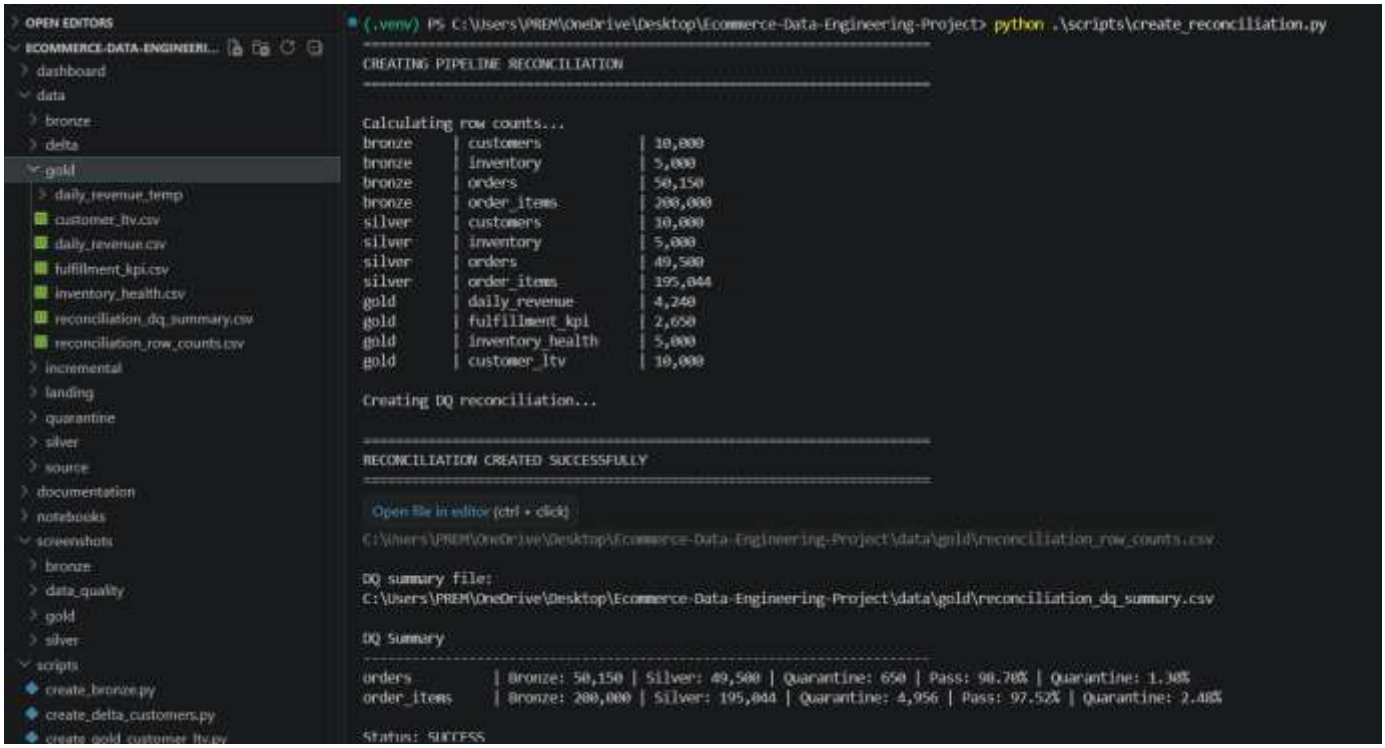
The final reconciliation files are stored in:

data/gold/

├── reconciliation_row_counts.csv

└── reconciliation_dq_summary.csv

The reconciliation process completed successfully and provides the final audit information required to verify the pipeline execution.



16. Azure Data Lake Storage Gen2

The project was extended to Azure Data Lake Storage Gen2 to provide cloud-based storage for the E-Commerce data pipeline.

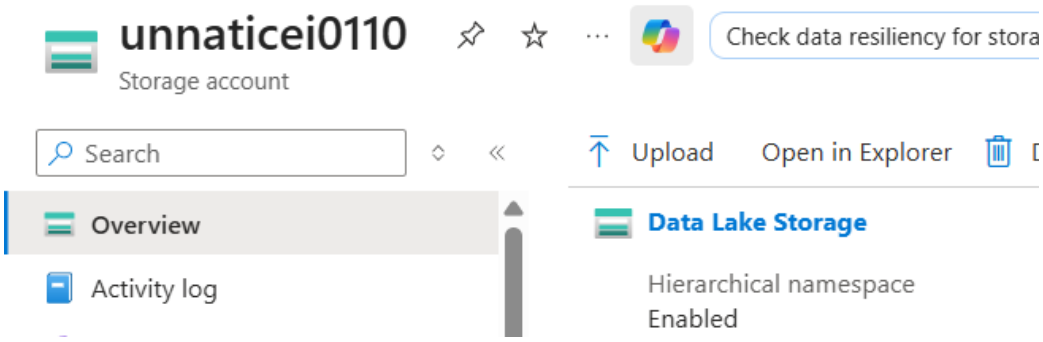
An existing Azure StorageV2 account was upgraded to **Azure Data Lake Storage Gen2 capabilities** by enabling hierarchical namespace functionality.

Before the upgrade, Azure validation was performed to identify incompatible storage features. The validation initially identified the following incompatible settings:

- containerDeleteRetentionPolicy
- blobDeleteRetentionPolicy

The corresponding container and blob soft-delete settings were disabled, after which the account successfully passed Azure's validation.

The Data Lake Gen2 upgrade was then completed successfully through the Azure Portal.



ADLS Gen2 Container

A private container named:

ecommerce-data

was created to store the project data.

The following data-layer structure was implemented:

ecommerce-data/

├── landing/

├── bronze/

├── silver/

├── gold/

└── quarantine/

Azure Data Organization

The project data was uploaded according to the pipeline architecture.

Landing Layer

Contains the original source datasets:

customers.csv

inventory.csv

orders.csv

order_items.csv

Bronze Layer

Contains the ingested datasets with Bronze metadata:

customers_bronze.csv

inventory_bronze.csv

orders_bronze.csv

order_items_bronze.csv

Silver Layer

Contains the validated and cleaned datasets:

customers/

  customers_silver_validated.csv

inventory/

  inventory_silver_validated.csv

orders/

  orders_silver_validated.csv

order_items/

  order_items_silver_validated.csv

Quarantine Layer

Contains records rejected during data-quality and referential-integrity validation:

orders/

  orders_quarantine.csv

order_items/

  order_items_quarantine.csv

Gold Layer

Contains the final analytical datasets:

daily_revenue.csv

fulfillment_kpi.csv

inventory_health.csv

customer_ltv.csv

reconciliation_row_counts.csv

reconciliation_dq_summary.csv

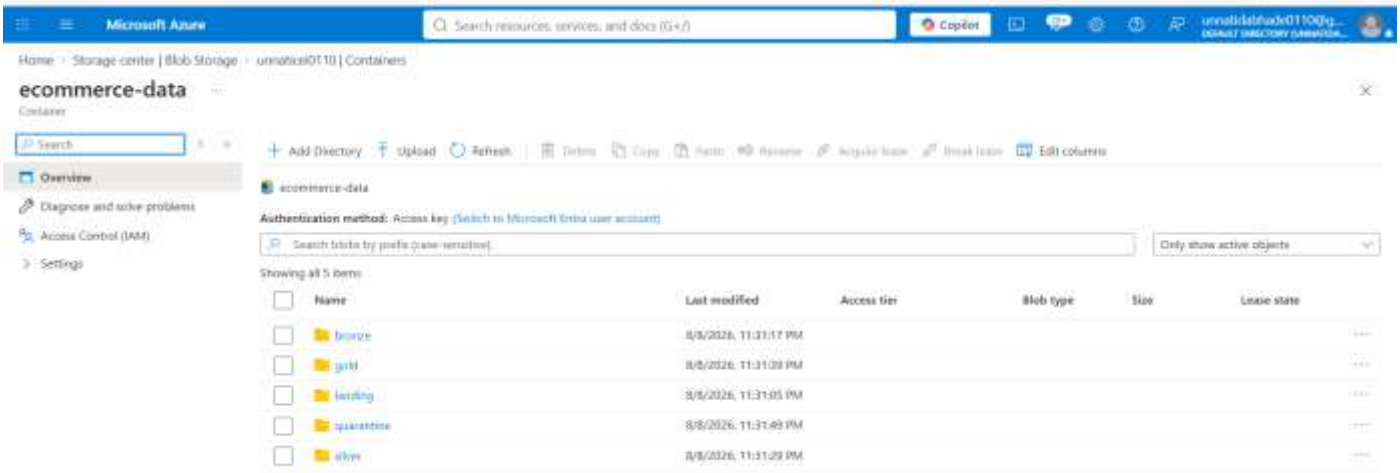
Azure Verification

The complete ADLS Gen2 structure was verified through the Azure Portal. All five pipeline layers and their corresponding files were successfully uploaded and confirmed.

This provides the project with a cloud-based data lake architecture that mirrors the local:

Landing → Bronze → Silver → Quarantine → Gold

pipeline.



17. Azure Databricks Integration

Azure Databricks was integrated with the ADLS Gen2 storage account to process the E-Commerce data using Apache Spark and PySpark.

The Databricks environment was used as the main processing engine for the cloud-based pipeline.

The processing architecture follows:

ADLS Gen2 → Databricks → Bronze → Silver → Gold → Reconciliation

The Databricks notebook was configured to access the ecommerce-data container and process the datasets stored in the different data layers.

The pipeline was executed using a Databricks cluster with Apache Spark.

18. Databricks Storage Authentication

Secure access to the ADLS Gen2 storage account was implemented using a Databricks Secret Scope.

The storage account key was stored securely under:

Secret Scope: azure-storage
Secret Key: storage-account-key

19. Databricks Pipeline Execution

The Databricks notebook was organized into sequential processing stages.

The pipeline was executed in the following order:

Landing

↓

Bronze

↓

Silver

↓

Data Quality / Quarantine

↓

Delta Processing

↓

Gold

↓

Reconciliation

Each stage reads the output generated by the previous stage and writes its results back to the corresponding ADLS Gen2 directory.

This provides clear separation between raw, processed, validated, and analytical data.

20. Bronze Processing in Azure Databricks

The Landing datasets were read from ADLS Gen2 using Spark.

The four datasets processed were:

- Customers
- Inventory
- Orders

- Order Items

During Bronze processing, ingestion metadata was added to each dataset:

- bronze_ingestion_timestamp
- source_file_name
- load_date

The Bronze datasets were written back to:

abfss://ecommerce-data@<storage-account>.dfs.core.windows.net/bronze/

The original Landing files were not modified.

This maintained the raw source data while creating a traceable Bronze processing layer.

21. Silver Processing in Azure Databricks

The Bronze datasets were loaded into Databricks and processed using PySpark transformations.

The Silver processing included:

- Data type conversion
- Null handling
- Duplicate removal
- Text standardization
- Date and timestamp conversion
- Email standardization
- Business-rule validation
- Referential-integrity validation

The validated datasets were written to the Silver layer in ADLS Gen2.

The resulting structure was:

silver/

├── customers/

├── inventory/

├── orders/

└── order_items/

Only trusted records were allowed to continue to downstream processing.

22. Delta Lake Processing

Delta Lake was used in the Databricks environment to provide transactional storage and incremental processing capabilities.

The main Delta use case was the **Customers** dataset because customers arrive as full daily snapshots.

A **Slowly Changing Dimension Type-1** approach was implemented.

The processing logic was:

- If customer_id already exists → update the existing customer record.
- If customer_id does not exist → insert the new customer record.
- Existing customer values are overwritten with the latest information.

The Delta implementation also provides transactional consistency for incremental processing.

23. Gold Processing in Azure Databricks

After Silver validation and Delta processing, the validated data was transformed into business-ready Gold datasets.

The following Gold datasets were generated:

gold/

├── daily_revenue.csv

├── fulfillment_kpi.csv

├── inventory_health.csv

├── customer_ltv.csv

├── reconciliation_row_counts.csv

└── reconciliation_dq_summary.csv

The Gold datasets were generated using Spark aggregations and joins across the validated Silver datasets.

Each Gold dataset represents a specific business analysis area.

24. Gold KPI Validation

After each Gold dataset was generated, KPI calculations were performed to validate the output.

Daily Revenue

The pipeline calculated:

- Total revenue
- Total orders

- Average order value
- Revenue by region
- Revenue by category

Fulfillment

The pipeline calculated:

- Delivery rate
- Cancellation rate
- Shipment rate
- Warehouse performance

Inventory

The pipeline calculated:

- Total SKUs
- Stockout count
- Below-reorder count
- Overstock count
- Inventory value

Customer LTV

The pipeline calculated:

- Total customers
- Active customers
- Average customer LTV
- Customer segments

The KPI results were compared with the generated Gold datasets to ensure the transformations completed correctly.

25. Reconciliation in Azure Databricks

A final reconciliation stage was executed after Gold processing.

The reconciliation verified the number of records processed through the different pipeline stages.

The following outputs were generated:

reconciliation_row_counts.csv

reconciliation_dq_summary.csv

The reconciliation compared:

Bronze → Silver → Quarantine → Gold

This confirmed that rejected records were separated from trusted records and that the expected datasets were successfully produced.

The reconciliation results also provide an audit trail for pipeline execution.

26. Azure Data Factory Orchestration

Azure Data Factory was used to automate the execution of the Databricks pipeline.

An ADF pipeline was configured to trigger the Databricks notebook after the required source data was available in the ADLS Gen2 Landing layer.

The orchestration flow was:

ADLS Gen2

↓

ADF Pipeline

↓

Databricks Notebook

↓

Bronze

↓

Silver

↓

Quarantine / Validation

↓

Delta Processing

↓

Gold

↓

Reconciliation

The pipeline was configured to run on a 6-hour schedule, as required by the project specification.

This removes the need for manually starting the Databricks notebook for every processing cycle.

27. End-to-End Azure Pipeline

The complete project was set up using Azure services.

First, the four CSV files were stored in **Azure Data Lake Storage Gen2**. The files were kept in the Landing layer.

Azure Data Factory was used to start and manage the data pipeline.

The data was then processed in **Azure Databricks**. The data went through the Bronze and Silver layers, where it was cleaned and checked.

Records with data problems were moved to the **Quarantine layer**. The correct records continued to the next step.

Delta Lake was used to manage customer updates using **SCD Type-1**.

Finally, the processed data was stored in the **Gold layer**. A reconciliation step was performed to check the final data.

The overall flow was:

ADLS Gen2 → ADF → Databricks → Bronze → Silver → Quarantine → Delta/SCD-1 → Gold → Reconciliation

This completed the end-to-end Azure data pipeline.

28. Dashboard – Executive Overview

A Power BI dashboard was created using the Gold-layer analytical datasets to provide a simple business view of the E-Commerce pipeline results.

The dashboard combines key metrics from the **Daily Revenue, Customer LTV, Fulfillment KPI, and Inventory Health** datasets.

Key Performance Indicators

The dashboard includes the following KPI cards:

- **Total Revenue** – overall revenue generated from the validated sales data.
- **Total Orders** – total number of orders included in the analytical dataset.
- **Total Customers** – total customers available for analysis.
- **Average Customer LTV** – average lifetime spending per customer.
- **Delivery Rate** – delivery performance calculated from fulfillment data.
- **Inventory Value** – total value of available inventory.

Revenue Analysis

Two charts were added to provide a quick view of revenue performance:

- **Revenue by Region** – compares total revenue across different regions.
- **Revenue by Category** – compares total revenue across product categories.

These visualizations allow business users to quickly identify the regions and product categories contributing the most revenue.

Dashboard Design

The dashboard was designed as an **Executive Overview** with KPI cards at the top and revenue analysis charts below.

The dashboard provides a simple business-facing view of the Gold datasets without requiring users to inspect the underlying CSV files.

The final dashboard was created in **Microsoft Power BI** and uses the Gold-layer CSV datasets as its data sources.

Dashboard outcome: The Gold analytical outputs were successfully converted into an interactive business dashboard for executive-level reporting and analysis.

29. Conclusion

The E-Commerce Data Engineering Pipeline was successfully developed as an end-to-end data processing solution following the required **Landing → Bronze → Silver → Quarantine → Gold** architecture.

The project started with the four provided source datasets: **Customers, Inventory, Orders, and Order Items**. The data was ingested into the Landing layer and then processed through the Bronze and Silver layers. Data-quality checks, deduplication, transformation, and referential-integrity validation were applied to improve data reliability. Invalid records were separated into the Quarantine layer instead of being passed to downstream analytics.

The Gold layer produced business-ready datasets for **Daily Revenue, Fulfillment KPI, Inventory Health, and Customer LTV**, along with reconciliation outputs for data auditing and validation.

The pipeline was also extended to **Azure Data Lake Storage Gen2, Azure Databricks, and Azure Data Factory**, providing a cloud-based architecture for storing, processing, and orchestrating the data pipeline. Customer updates were designed around the required **SCD Type-1** approach.

Finally, the Gold datasets were connected to **Power BI** to create an Executive Overview dashboard containing important business KPIs and revenue analysis.

Overall, the project demonstrates how raw e-commerce data can be transformed into **clean, validated, organized, and business-ready information** using a modern data engineering architecture.